



MODULE NAME:	MODULE CODE:
INFORMATION SYSTEMS 3D	INSY7314/w
APPLICATION DEVELOPMENT SECURITY	APDS7311/w

ASSESSMENT TYPE: POE (PAPER AND MARKING RUBRIC)

TOTAL MARK ALLOCATION: 300 MARKS

TOTAL HOURS: A minimum of 45 HOURS is suggested to complete this assessment

By submitting this assignment, you acknowledge that you have read and understood all the rules as per the terms in the registration contract, in particular the assignment and assessment rules in The IIE Assessment Strategy and Policy (IIE009), the intellectual integrity and plagiarism rules in the Intellectual Integrity Policy (IIE023), as well as any rules and regulations published in the student portal.

INSTRUCTIONS:

1. ***No material may be copied from original sources, even if referenced correctly, unless it is a direct quote indicated with quotation marks. No more than 10% of the assignment may consist of direct quotes.***
2. ***Please ensure that you submit your assignment through Turnitin.***
3. ***The assignment must be pushed to GitHub.***
4. ***Assignments must be typed unless otherwise specified.***
5. ***Follow all instructions on the PoE cover sheet.***
6. ***This is a group assignment. Please allow for a maximum of 4 participants.***

Referencing Rubric

Providing evidence based on valid and referenced academic sources is a fundamental educational principle and the cornerstone of high-quality academic work. Part of achieving this quality is referencing in a way that is consistent and congruent with the requirements of the referencing style being used.

Therefore, inconsistent and/or incongruent referencing will result in a penalty of **a maximum of ten percent being deducted from the overall percentage** awarded to your assessment submission.

Please note that **evidence of plagiarism** in the form of copied or unreferenced work, absent reference lists, or exceptionally poor referencing **may result in action being taken in accordance with The IIE's Intellectual Integrity and Property Rights Policy (IIE023)**. Similarly, **evidence of excessive AI usage may result in action being taken in accordance with The IIE's Student Conduct, Discipline and Safety Policy (IIE015)**.

Markers are required to provide feedback to students by **circling/underlining the information in the table below that best describes the student's work and by adding constructive commentary where appropriate**. The examples provided are not exhaustive but illustrate the errors.

Deductions

- Where the student's work contains **five or more errors** aligned to the **minor errors column** below, **deduct 5% from the overall percentage**.
- Where the student's work contains **five or more errors** aligned to the **major errors column** below, **deduct 10% from the overall percentage**.
- Where **both minor and major errors** (e.g. two minor and three major, etc.) are present, **deduct 10% only** (and not 5% or 15%) from the overall percentage.

Required: Consistent and congruent referencing	Minor errors Deduct 5% from overall percentage. Example: if the response receives 70%, deduct 5%. The final mark is 65%.	Major errors Deduct 10% from the overall percentage. Example: if the response receives 70%, deduct 10%. The final mark is 60%.
Consistency - The correct referencing style for the discipline – i.e., either Harvard, OR APA (for Psychology), OR Law, OR IEEE (for ICT/Engineering) – has been used consistently for all in-text references and in the bibliography/reference list. Concepts and ideas that are quoted and/or paraphrased are referenced consistently throughout. Position of the in-text reference: an in-text reference is positioned consistently where appropriate for every quote and paraphrase.	Minor inconsistencies: - The referencing style used is generally consistent with what is required, but there are one or two changes/errors in the format of in-text referencing and/or in the bibliography/reference list. - For example, page numbers for direct quotes in-text have been provided for one source, but not in another. Or, two book chapters in the bibliography/reference list have been referenced in two different formats. Or, the publication year has been placed after the author name in one bibliography/reference list entry, and after the source title in another, etc. - Concepts and ideas in quotes and/or paraphrases are typically referenced, but a full in-text reference is missing or incomplete from one or two small sections of the work. - Position of the references: in-text references are only given at the beginning and/or end of every paragraph.	Major inconsistencies: - Poor and wholly inconsistent referencing style used in-text and/or in the bibliography/reference list. - Multiple referencing styles for the same source types have been used. - For example, the format for direct quotes in-text and/or book chapters in the bibliography/reference list and/or year of publication in the bibliography/reference list is different across multiple instances. - Concepts and ideas in quotes and/or paraphrases are haphazardly referenced in-text. - Position of the references: in-text references are only given at the beginning or end of large sections of work.
Feedback on referencing consistency:		
Congruency Each source reflected within in-text references is included accurately in the bibliography/reference list. - All bibliography/reference list entries are in the required order for the referencing style used (e.g. alphabetical, alphabetical under subheadings, numerical). - All direct quotes and paraphrases have been integrated appropriately into the text using introductory phrases, accurate grammar, etc.	Minor incongruities: - There is largely a match between the sources presented in-text and those in the bibliography/reference list, but one or two sources that appear in-text do not appear in the bibliography/reference list, or vice versa. Or key source information is missing from one or two in-text references or bibliography/reference list entries only (e.g. publication year, city of publication, URL date accessed, etc.). - There is a clear and largely accurate ordering of sources in the bibliography/reference list as required by the referencing style used, but with one or two references out of order. - An attempt has been made for source integration into the text using appropriate introductory phrases and grammar, but one or two quotes or paraphrases do not flow as clearly or logically within the sentence structure as they could.	Major incongruities: - No relationship/several incongruities between the in-text referencing and the bibliography/reference list. - For example, multiple sources are included in-text, but not in the bibliography, and/or vice versa. Key source information is missing from multiple in-text references and/or reference list entries. A URL link, rather than the actual reference, is provided in the bibliography. Sources are repeated in the reference list, etc. - Most sources are listed in a haphazard order throughout the bibliography/reference list. - Few to no appropriate introductory phrases or rules of grammar have been applied, and many direct quotes and/or paraphrases feel disconnected from the flow of the text.
Feedback on referencing congruency:		
Overall feedback on referencing, with suggested improvements:		

Background

You are a software development team tasked with designing and implementing **HustleHub+**, a secure freelance marketplace platform. The purpose of the platform is to allow freelancers to advertise services and clients to browse and book those services.

In addition to standard marketplace functionality, the system must record financial transactions generated through bookings and provide users with an indication of income earned and estimated tax obligations.

The platform processes sensitive information including user credentials, transactional records, and income-related data. As such, the system must be designed and implemented with security as a primary concern, rather than as an afterthought.

This POE requires you to apply knowledge and skills across backend development, frontend integration, API security, testing, and DevSecOps practices. Each part of the POE builds on the previous one and contributes toward a fully functional, secure, and testable system.

Development Approach

The POE follows an incremental development approach:

- Part 1: Secure backend foundations
- Part 2: Full-stack application development
- Part 3: DevSecOps, monitoring, and finalisation

Students are expected to refine and improve their system at each stage. Work submitted in later parts must demonstrate clear progression and improvement based on earlier submissions.

Overall Requirements

By the final submission, the **HustleHub+** system must support:

- User registration and authentication using secure practices
- Role-based access control for Clients, Freelancers, and Admin users
- Freelancer management of gigs
- Client browsing and booking of gigs
- Creation of transaction records based on bookings (payments are simulated)
- Tracking of income for Freelancers
- Calculation and display of estimated tax values for Freelancers
- A user-facing dashboard displaying relevant financial information for Freelancers
- Implementation of security controls including validation, sanitisation, and access restrictions
- Logging of key system events
- Containerisation of the backend and frontend applications.
- A working CI/CD pipeline that includes both the backend and frontend applications, including:
 - Automated backend and frontend unit testing.
 - Automated API endpoint testing.
 - Automated security scans.

Part 1 - Secure Foundations**(Marks: 100)**

At the end of this specific part, students should be able to:

- *Explain and implement the architecture of a web-based backend application using Node.js and Express.*
- *Implement secure user authentication using password hashing and JSON Web Tokens (JWT).*
- *Apply fundamental application security practices, including input validation, HTTPS configuration, and secure error handling.*

In this part, you are required to design and implement the foundational backend for **HustleHub+**. The focus is on establishing a secure API that supports user registration and authentication. All written explanations must be included within the project README. No separate documents are required for this part.

Requirements

Here are the requirements for this assessment:

- An architecture diagram outlining the components of the overall system, including security features and system boundaries. The architecture diagram must reflect the MERN architecture.
- The backend API, implemented using Node.js and Express.
- Support for user registration and login functionality. At this stage, user data may be stored locally using an in-memory structure or file-based storage. A database will be required later.
- Passwords must be securely stored using a hashing algorithm. Under no circumstances may plain-text passwords be retained.
- The login processes must utilise JWT to identify authenticated users in subsequent requests. JWT must be used to protect API routes beyond login and must be validated on each protected request.
- The application must be served over HTTPS using a locally configured SSL certificate.
- All user input must be validated before processing. Invalid or malicious input must not be accepted.
- Error responses must be controlled and must not expose internal system details such as stack traces, file paths, or configuration values.
- The README must include a clear explanation of the system, the intended users, the structure of the backend, and the security decisions that have been made.

This includes a discussion of password hashing, token-based authentication, input validation, and the importance of HTTPS.

- API endpoints must be tested using Postman. The submission must include a Postman collection demonstrating successful registration and login requests, as well as appropriate handling of invalid scenarios.
- A demonstration video must be recorded showing the API running, successful user registration, and login with token generation

Submission requirements

- GitHub repository link
- README containing all required documentation
- Postman collection
- Screenshots of API responses
- Demonstration video link

Part 2 - Secure Stack

(Marks: 100)

At the end of this specific part, students should be able to:

- *Develop a full-stack MERN application with integrated frontend, backend, and database components.*
- *Implement and enforce authentication and authorisation mechanisms, including role-based access control (RBAC).*
- *Apply application security techniques, including validation, sanitisation, rate limiting, security headers, and testing of both backend and frontend components.*

In this part, you are required to extend your Part 1 backend into a full-stack application. The system must now support marketplace functionality, including gig creation, browsing, and booking.

Security must be applied throughout the application. You are expected to demonstrate that sensitive operations are protected and that access to data is controlled appropriately based on a user's role.

Requirements

Here are the requirements for this assessment:

- The backend must support gig management functionality, where freelancers are able to create, update, view, and delete their own gigs. Users must not be able to modify, delete, or access resources that do not belong to them.
- A React frontend must be developed to support user interaction.
- The interface must allow users to register, log in, browse gigs, and perform bookings. The application must handle invalid inputs gracefully and must not expose sensitive information through the user interface.
- Clients must be able to view available gigs and initiate bookings. The booking process does not require real payment integration; instead, a simulated confirmation is sufficient. However, each booking must result in a transaction record being created and associated with the relevant users.
- Freelancers must be able to create their listings and view their bookings. The system must track income generated from bookings and associate this income with the correct freelancer.
- Security controls must be clearly implemented. All protected routes must require authentication via JWT. Role-based access control must be enforced to ensure that only authorised users can perform specific actions. Input data must be sanitised to prevent injection and scripting attacks.
- Rate limiting must be applied to sensitive endpoints, particularly those related to authentication and booking. Security headers must be configured to protect the application against common web-based attacks, including the use of a Content Security Policy.
- Testing must be extended to include both backend and frontend components. The Postman collection must be expanded to cover key endpoints, and Newman must be used to execute these tests. The frontend must include basic automated tests that demonstrate component rendering and user interaction.
- The README must be updated to reflect the expanded system. It must include a description of the application features, security measures, and instructions for running both the backend and frontend applications. Testing procedures must also be documented.
- A demonstration video must be recorded showing the complete application flow, including authentication, gig creation, booking creation and viewing, and transaction recording.

Submission requirements

- Updated GitHub repository

- Updated README
- Backend and frontend code
- Postman collection and Newman execution evidence
- Frontend test evidence
- Screenshots of application
- Demonstration video link

POE – DevSecOps, Monitoring and Finalisation

(Marks: 100)

At the end of this specific part, students should be able to:

- *Implement DevSecOps practices using CI/CD pipelines to automate testing, building, and quality checks.*
- *Apply deployment practices, including containerisation of applications using Docker.*
- *Implement logging and monitoring mechanisms and evaluate the effectiveness of security controls within an application.*

In this final part, you are required to complete the **HustleHub+** system by adding financial insights, DevSecOps practices, and monitoring capabilities. The system must demonstrate readiness for deployment and must reflect a mature implementation of security principles.

Requirements

Here are the requirements for this assessment:

- The system must include a tax estimation component that calculates an approximate tax value based on income generated. The logic used for this calculation must be clearly explained. The frontend must display income and tax information to freelancers in a structured and visually appealing manner, such as through charts or summaries.
- Students must implement a CI/CD pipeline using GitHub Actions. The pipeline must execute automatically and include processes for installing dependencies, running backend tests, executing frontend tests, and building the backend and frontend applications. The pipeline must fail when tests or checks do not pass.
- Static code analysis must be performed in the pipeline using an appropriate tool. The submission must include evidence of the analysis results and a brief explanation of what the results indicate.

- The backend API and frontend application must be containerised using Docker with orchestration managed using Docker Compose. The submission must demonstrate that the application stack can run within a containerised environment.
- Logging must be implemented to capture key system events, including login attempts, bookings, transactions, and errors. The README must explain what is logged and why these logs are important for monitoring and security.
- Students must implement at least two additional security features beyond the minimum requirements. These features must be functional and clearly documented.
- A final security review must be included in the README, explaining how the system addresses common threats such as injection, cross-site scripting, authentication weaknesses, and access control issues.
- The final README must serve as a comprehensive documentation of the system, including setup instructions, feature descriptions, security implementation, testing, pipeline configuration, logging, and demonstration evidence.
- A demonstration video must be recorded showing the complete application flow, including authentication, gig creation, booking, and transaction recording.

Submission requirements

- Final GitHub repository
- Final README
- Pipeline configuration files
- Static analysis result
- Both Dockerfiles inside the relevant folders (API and client) with Docker Compose for orchestration.
- Logging evidence
- Demonstration video

Appendix A - Marking Rubrics

MODULE NAME:	MODULE CODE:
INFORMATION SYSTEMS 3D	INSY7314/w
APPLICATION DEVELOPMENT SECURITY	APDS7311/w

STUDENT NAME:
STUDENT NUMBER:

PART 1				
Marking Criteria	Does not meet the required standard	Meets the required standard	Partially exceeds the required standard	Greatly exceeds the required standard
README (System Architecture, Security) 20 Marks	• README missing or incomplete. • System purpose unclear. • No meaningful explanation of architecture or security.	• Basic description of system and roles. • Mentions hashing/JWT/HTTPS but lacks clarity or correctness.	• Clear explanation of system, backend structure, and security decisions. • Correct explanation of hashing, JWT flow, validation, HTTPS.	• Well-structured, professional documentation. • Clearly explains architecture, request flow, token usage, validation approach, and security rationale. • Demonstrates understanding of why each decision was made.
	0-5 Marks	6-10 Marks	11-15 Marks	16-20 Marks
API Functionality (Register/Login) 20 Marks	• API fails to run or endpoints missing. • Responses incorrect or inconsistent.	• API runs. Register/login partially working. • Some incorrect responses or missing edge-case handling.	• Register and login fully functional. • Correct status codes, JSON responses, and basic error handling implemented.	• Robust implementation. • Handles edge cases (duplicate users, invalid login, missing fields). • Consistent response structure and clean routing/controller separation.
	0-5 Marks	6-10 Marks	11-15 Marks	16-20 Marks

PART 1				
Marking Criteria	Does not meet the required standard	Meets the required standard	Partially exceeds the required standard	Greatly exceeds the required standard
API Functionality (Password Security & JWT) 20 Marks	• Passwords stored in plain text or insecurely. • No JWT or incorrect implementation.	• Password hashing implemented but inconsistent or weak usage. • JWT generated but not properly structured or reused.	• Passwords securely hashed. • JWT includes user identification and is returned on login.	• Hashed passwords verified correctly. • JWT contains appropriate payload (id/role), secure secret usage, token structure consistent, ready for route protection.
	0-5 Marks	6-10 Marks	11-15 Marks	16-20 Marks
HTTPS Implementation 10 Marks	• No HTTPS configured.	• Attempt made but not working correctly or inconsistently applied.	• HTTPS correctly configured locally and API runs securely.	• HTTPS correctly configured and demonstrated. • Shows clear understanding of secure communication and its importance.
	0-3 Marks	4-5 Marks	6-7 Marks	8-10 Marks
Input Validation & Error Handling 10 Marks	• No validation. • Errors expose internal details (stack traces, raw errors).	• Basic validation present but inconsistent. • Some unsafe responses.	• Validation applied to key inputs. • Errors handled safely with controlled responses.	• Comprehensive validation. Handles missing, invalid, and malicious input. • Error responses are consistent, safe, and user-friendly.
	0-3 Marks	4-5 Marks	6-7 Marks	8-10 Marks
Code Structure 10 Marks	• Code poorly structured. • Logic mixed in single files. • Difficult to follow.	• Basic structure present but inconsistent separation of concerns.	• Clear structure with routes, controllers, and models separated.	• Clean, modular architecture. Consistent naming, reusable components, and maintainable structure.
	0-3 Marks	4-5 Marks	6-7 Marks	8-10 Marks

PART 1				
Marking Criteria	Does not meet the required standard	Meets the required standard	Partially exceeds the required standard	Greatly exceeds the required standard
Postman Testing 10 Marks	Application incomplete, major features missing or broken.	Core features implemented but inconsistent or partially functional.	All required features functional (gigs, bookings, transactions, income).	- Fully functional system. Smooth flow from authentication, gig, booking, transactions, to income tracking. - No major issues.
	0-3 Marks	4-5 Marks	6-7 Marks	8-10 Marks
Incorrect GitHub used or not GitHub Used -5 Marks	No GitHub repository submitted	- GitHub repository submitted with limited commit history and limited evidence of collaboration. - No README.md file included	- GitHub repository submitted with good commit history (25 commits) - Evidence of collaboration (multiple contributors). - README.md file included but not formatted in Markdown.	- GitHub repository submitted with excellent commit history (25+ commits and well-structured commit messages). - Evidence of collaboration (multiple contributors). - README.md file included and perfectly formatted in Markdown.
	-5 Marks	-4 or -3 Marks	-2 or -1 Marks	No deduction
PART 1 = 100 Marks				

PART 2				
Marking Criteria	Does not meet the required standard	Meets the required standard	Partially exceeds the required standard	Greatly exceeds the required standard
Application Functionality 20 Marks	Application incomplete, major features missing or broken.	Core features implemented but inconsistent or partially functional.	All required features functional (gigs, bookings, transactions, income).	- Fully functional system. Smooth flow from authentication, gig, booking, transactions, to income tracking. - No major issues.
	0-5 Marks	6-10 Marks	11-15 Marks	16-20 Marks
RBAC Enforcement 15 Marks	- Roles not enforced. - Users can access or modify unauthorised data.	Basic role checks present but inconsistent.	Role restrictions applied correctly to most endpoints.	Strong RBAC enforcement across API. Users cannot access or modify resources they do not own. Clear separation of client/freelancer/admin capabilities.
	0-4 Marks	5-8 Marks	9-12 Marks	13-15 Marks
API Security (JWT, Validation, Sanitisation) 15 Marks	Security missing or ineffective.	Basic JWT and validation applied but inconsistent.	- Good protection across endpoints. - Inputs validated and sanitised.	- Strong, consistent security across all endpoints. - Proper token validation, sanitisation, and secure handling of user input.
	0-4 Marks	5-8 Marks	9-12 Marks	13-15 Marks
Rate Limiting 10 Marks	Not implemented.	Basic rate limiting applied to limited endpoints.	Applied correctly to login and booking endpoints.	Well-configured with clear limits and meaningful responses when exceeded.
	0-3 Marks	4-5 Marks	6-7 Marks	8-10 Marks

PART 2				
Marking Criteria	Does not meet the required standard	Meets the required standard	Partially exceeds the required standard	Greatly exceeds the required standard
Security Headers (Helmet + CSP) 10 Marks	Not implemented.	Helmet used but CSP incomplete or weak.	Helmet and CSP correctly configured.	Strong configuration. CSP appropriately restricts resources and protects against common attacks.
	0-3 Marks	4-5 Marks	6-7 Marks	8-10 Marks
React Frontend 10 Marks	Incomplete or not functional.	Basic UI with limited usability.	Functional UI with logical flow and acceptable usability.	- Clean, consistent, user-friendly interface. - Handles invalid input gracefully - Integrates well with API.
	0-3 Marks	4-5 Marks	6-7 Marks	8-10 Marks
Backend Testing (Postman + Newman) 10 Marks	No testing or Newman not used.	Basic Postman tests. Limited Newman evidence.	Newman runs successfully for key endpoints.	Structured test collection with clear success/failure cases and reliable Newman execution.
	0-3 Marks	4-5 Marks	6-7 Marks	8-10 Marks
Frontend Testing 5 Marks	No tests.	One basic test present.	Rendering + interaction test implemented.	Meaningful tests validating component behaviour and user interaction.
	0 Marks	1-2 Marks	3-4 Marks	5 Marks
README 5 Marks	Poor or missing.	- Basic documentation. - Does not reflect any changes made in this version of the project.	- Clear and useful. - Reflects some changes made in this version of the project.	- Well-structured and complete. - Reflects all changes made in this version of the project.
	0 Marks	1-2 Marks	3-4 Marks	5 Marks

PART 2				
Marking Criteria	Does not meet the required standard	Meets the required standard	Partially exceeds the required standard	Greatly exceeds the required standard
Incorrect GitHub used or not GitHub Used -5 Marks	No GitHub repository submitted	- GitHub repository submitted with limited commit history and limited evidence of collaboration. - No README.md file included	- GitHub repository submitted with good commit history (50 commits) - Evidence of collaboration (multiple contributors). - README.md file included but not formatted in Markdown.	- GitHub repository submitted with excellent commit history (50+ commits and well-structured commit messages). - Evidence of collaboration (multiple contributors). - README.md file included and perfectly formatted in Markdown.
	-5 Marks	-4 or -3 Marks	-2 or -1 Marks	No deduction
PART 2 = 100 Marks				

POE				
Marking Criteria	Does not meet the required standard	Meets the required standard	Partially exceeds the required standard	Greatly exceeds the required standard
Final Features (Tax + Income Dashboard) 20 Marks	Missing or incorrect implementation.	Basic tax and income calculation and display.	Functional tax logic and income dashboard with correct values.	- Clear, accurate calculations. - Well-presented financial insights with meaningful visualisation.
	0-5 Marks	6-10 Marks	11-15 Marks	16-20 Marks
CI/CD Pipeline (Backend + Frontend) 20 Marks	Missing or not working.	Basic pipeline configured but limited checks.	Pipeline runs backend & frontend tests and build steps.	- Robust pipeline. - Executes a variety of security tests - Builds entire stack. - Correctly fails when checks do not pass.
	0-5 Marks	6-10 Marks	11-15 Marks	16-20 Marks
Static Code Analysis 10 Marks	No code analysis.	- Tool used but unclear results. - Tool not integrated correctly.	Results shown but unattended issues	- Results shown and no unresolved security issues. - Fixes applied are explained in README.
	0-3 Marks	4-5 Marks	6-7 Marks	8-10 Marks
Docker (Backend + Frontend) 10 Marks	No containerisation	Partial implementation to container either frontend or backend, but not both.	Both applications containerised and runnable.	- Fully working containers with clear instructions. - Both containers running in the pipeline.
	0-3 Marks	4-5 Marks	6-7 Marks	8-10 Marks
Logging 10 Marks	No logging.	Basic logs (console only).	Logs for key events (login, booking, transaction, error).	- Evidence of comprehensive logging using a cloud-based logging platform. - Clear purpose and explanation in README.
	0-3 Marks	4-5 Marks	6-7 Marks	8-10 Marks

POE				
Marking Criteria	Does not meet the required standard	Meets the required standard	Partially exceeds the required standard	Greatly exceeds the required standard
Own Security Features 15 Marks	Missing or not functional.	Basic features with limited value.	Functional features integrated into system.	Useful, well-integrated, and clearly enhance system security.
	0-4 Marks	5-8 Marks	9-12 Marks	13-15 Marks
Security Review in README 10 Marks	Missing or superficial review.	Basic discussion of security.	Good explanation of implemented controls.	Clear mapping between threats and implemented protections.
	0-3 Marks	4-5 Marks	6-7 Marks	8-10 Marks
README Update 5 Marks	Poor or missing.	- Basic documentation. - Does not reflect any changes made in this version of the project.	- Clear and useful. - Reflects some changes made in this version of the project.	- Well-structured and complete. - Reflects all changes made in this version of the project.
	0 Marks	1-2 Marks	3-4 Marks	5 Marks
Incorrect GitHub used or not GitHub Used -5 Marks	No GitHub repository submitted	- GitHub repository submitted with limited commit history and limited evidence of collaboration. - No README.md file included	- GitHub repository submitted with good commit history (75 commits) - Evidence of collaboration (multiple contributors). - README.md file included but not formatted in Markdown.	- GitHub repository submitted with excellent commit history (75+ commits and well-structured commit messages). - Evidence of collaboration (multiple contributors). - README.md file included and perfectly formatted in Markdown.
	-5 Marks	-4 or -3 Marks	-2 or -1 Marks	No deduction
POE = 100 Marks				

[TOTAL MARKS: 300]